What Can You Know on Launch Day?

Leakage, Horizons and Calibration in Predicting Google Play Success

Matin Nikoomanzar
University of California, Los Angeles

October 2026

Abstract

I revisit a model that predicted which new Google Play apps reach 100K installs. It scored an AUC of 0.862 on a 2018 to 2019 temporal split, but most of its inputs (monetization, size, privacy policy, content rating) were read off a June 2021 snapshot, years after launch, and its outcome gave older apps more time to grow. I build a feature availability table, keep only inputs that were known on launch day, and redefine success so every app gets the same horizon. Ranking apps by whether they end up in the top 5% of their launch month, the launch-time model scores 0.836 (95% CI 0.831 to 0.841) against 0.861 for the old feature set on the same split, with a calibration slope of 0.92. For absolute horizons I treat the single snapshot as current status survival data, estimate time to 10K, 100K and 1M installs with the NPMLE and a parametric (cloglog) model on launch covariates, and validate it forward in time. Pooled over launch cohorts, 3.7% of apps reach 100K installs within a year of launch, while the raw rate for apps 11 to 13 months old is 3.45%, so the pooled curve hides some cohort drift. The model is well calibrated when held-out launch months are interleaved with training months and under-predicts when it is fit on older launches only. I also give split conformal intervals with checked coverage, which are wide, and I bound the monetization associations with E-values instead of reading them as effects.

1 Introduction

App stores are a good place to study winner-take-most markets. On Google Play a tiny share of apps collect almost all installs, so the practical question for a developer or an early investor is whether you can tell, on launch day, which apps will be in that group. An earlier version of this project answered yes, with an AUC of 0.862. There are two problems with that answer. The inputs included things nobody could know on launch day, and the outcome gave apps released in early 2018 more than twice as long to grow as apps released in late 2019.

This paper is the rebuild. The research question is narrow on purpose:

> Using only information available on an app's launch day, how well can you rank and give calibrated probabilities for its eventual success, measured on a horizon that is the same for every app?

The question can fail. If launch-day facts carried no signal, the AUC would be near 0.5 and the honest conclusion would be that the old model only worked because it peeked. That is not what happens. The gap between the two numbers, and where it comes from, is the main result.

2 Data

The data is a scrape of 2,312,944 Google Play listings from June 15 and 16, 2021 [Prakash, 2021]. After dropping rows with no release date, install count or category I keep 2,241,891 apps from 736,124 developers. Each app is seen exactly once. Install counts (`Maximum Installs`) are cumulative and close to exact. Only apps still listed in June 2021 are present, so failed apps are under-represented and more so for older launch years. I come back to this in Section 9. 21.6% of apps were never updated after their release date. The old analysis used the 815,231 apps released in 2018 and 2019. The live simulator was trained on 799,763 of them (those with a known size and one of the four main content ratings), which is where the two app counts on the project website come from.

3 What was known on launch day

Table 1 summarizes the availability table in `docs/method.md`. Every input any model uses is tagged as known at launch, known only in the 2021 snapshot, or only defined after launch, together with a judgment of whether success can change it. A test fails if anything not tagged "launch" enters a launch-time model. A second test shuffles each app's own installs, ratings, monetization flags, size and update fields and checks that its launch features do not move. A third changes the installs, in-app purchase flag and title of apps released after a cutoff and checks that no launch feature of an earlier app moves. None of these tests can say whether a tagged "launch" field was edited later, which is why the table has an "editable" row.

Group	Examples	First known
Launch, fixed	release date, package name, developer's earlier releases	launch day
Launch, editable	category, app title (the title is the weaker case, since developers rewrite it for search)	launch day (2021 value used)
Snapshot	price, ads, in-app purchases, size, minimum Android, content rating, privacy policy, website	June 2021
After launch	ratings, rating count, editors' choice, last update, earlier apps' installs	June 2021

Table 1: Feature availability. The old model used eight snapshot fields plus category and developer app count.

The developer app count in the old model was built correctly in time (it ranks a developer's apps by release date), but it only counts apps that survived to 2021, and its labels in the write-up were off by one. A developer's *track record*, meaning how many earlier apps became hits, looks like launch information but is not, because those apps' installs were also counted in 2021, after the launch being predicted.

4 A consistent horizon

Ranking target. An app is a hit if its installs are in the top 5% of all apps released in the same calendar month. Every app is compared only with apps that had the same time to grow. The base rate is 5% in every month by construction.

Absolute target. App i is observed once, at age a_i, with $\delta_i = 1\{T_i \le a_i\}$, where T_i is the time to reach N installs. This is current status data. Without covariates the NPMLE of $F(t) = P(T \le t)$ is the isotonic regression of δ on a [Groeneboom and Wellner, 1992]. It converges at rate $n^{1/3}$, and the plain bootstrap is known to fail for cube-root estimators of this kind [Sen et al., 2010], so I use subsampling with the cube-root rate [Politis and Romano, 1994]. With launch covariates x I fit

$$P(T \le t \mid x) = G(\alpha + \beta \log t + x^\top \gamma), \tag{1}$$

where G is the complementary log-log (Weibull proportional hazards), probit (log-normal AFT) or logistic (log-logistic AFT) link. Each implies a time ratio $\exp(-\gamma_j/\beta)$ for covariate j. The model assumes no cohort effect, so given x the time to milestone does not depend on launch date. One snapshot cannot test that in sample, because age and launch date are the same thing, so I test it out of sample.

5 Ranking results

All ranking models train on 2018 launches (334,566 apps) and are scored on 2019 launches (480,665 apps). Confidence intervals come from 500 bootstrap draws that resample whole developers.

Model	AUC [95% CI]	Brier	Cal. slope	Top 10% finds
Category rate	0.726 [0.719, 0.733]	0.0453	0.97	31%
Developer history rate	0.593 [0.585, 0.602]	0.0472	1.19	13%
Category × developer	0.750 [0.743, 0.757]	0.0448	1.02	33%
Launch, strict (no category or title)	0.711 [0.704, 0.718]	0.0454	0.86	31%
Launch, spline logistic	0.825 [0.820, 0.830]	0.0425	0.99	45%
Launch, gradient boosting	**0.836** [0.831, 0.841]	0.0417	0.92	47%
Old features, logistic	0.845 [0.839, 0.849]	0.0405	1.00	50%
Old features, gradient boosting	0.861 [0.857, 0.865]	0.0395	0.90	53%

Table 2: 2019 launches, top 5% of launch month. The old feature set uses 2021 snapshot fields and is shown for comparison only. Paired difference old minus launch: 0.025 [0.021, 0.029].

Where the old AUC came from. On the old outcome (100K by June 2021) the old feature set scores 0.861 in my refit, close to the 0.862 in the original report. Scored within launch month, which removes the free credit for being older, it scores 0.861. On the horizon-consistent target the ablation in Figure 1 gives 0.857 for the old features, 0.816 without monetization, 0.748 with every snapshot field removed, and 0.830 for the launch features. Dropping the title features from the launch set gives 0.783 and dropping category gives 0.779. The title is a 2021 value that developers edit, so part of its signal may come from changes made after launch. Adding the three monetization flags back to the launch set gives 0.858, all snapshot fields 0.876, earlier apps' 2021 installs 0.893, and the 2021 ratings and update fields 0.984, which shows how fast a model can look brilliant when it is allowed to read the outcome. Removing every snapshot field from the old features costs 30% of the old model's lift over an AUC of 0.5, but that comparison is against a weak model (category and developer app count only). Against the full launch model the gap is 8% of that lift, because the launch set also has developer history features that the old set lacked. Scoring within launch month leaves the old AUC unchanged (0.861 against 0.861), so among 2019 launches the unequal horizon did not inflate the ranking number. That check covers twelve launch months and cannot rule out

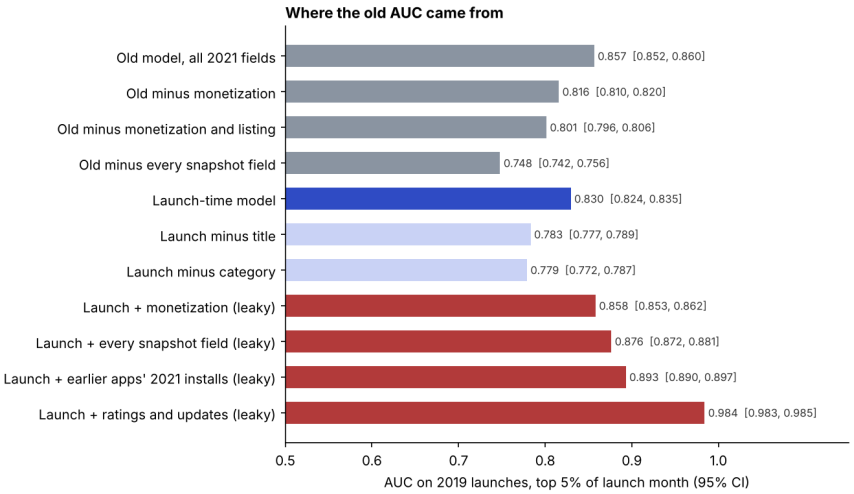


Figure 1: The leakage ablation on 2019 launches, all fit on the same 135,263 training apps (a 40% sample of training developers).

inflation over the 2016 to 2020 range. The unequal horizon does change the 100K rate, which falls with launch year in the data.

Calibration and stability. The launch model's calibration slope is 0.92 and its intercept −0.05 (Figure 2). Three refits on different halves of the training developers give AUC 0.831 with standard deviation 0.0009 (old features: 0.859, 0.0007).

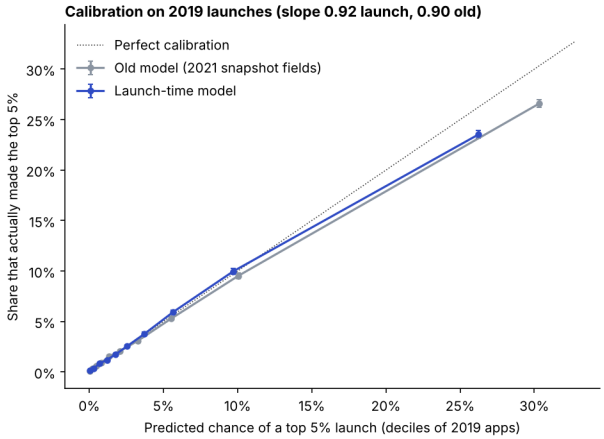


Figure 2: Reliability on 2019 launches by decile of predicted probability, with Wilson intervals.

Shift. Trained on 2016 and 2017 launches, the launch model scores 0.824, 0.825 and 0.813 on 2018, 2019 and 2020 launches (old features: 0.864, 0.854, 0.824). The top 1% and top 10% targets give 0.874 and 0.788.

Errors. The model's top 10% contains 47% of all hits, and 23.5% of the apps it flags are hits. Games make up more of the caught hits because their base rate is higher, and ranking within games

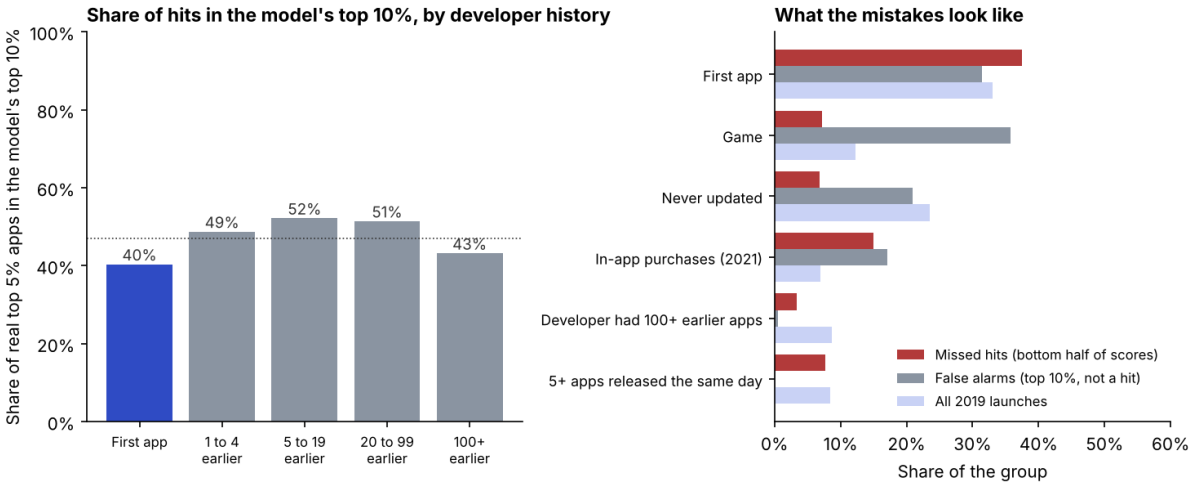


Figure 3: Error analysis on 2019 launches. Left: share of real hits that land in the model's top 10%, by developer history. Right: who the missed hits and false alarms are. 2021-only fields appear here as descriptions, never as inputs.

is a little worse than within other apps. Figure 3 shows where it fails.

6 Time to milestones

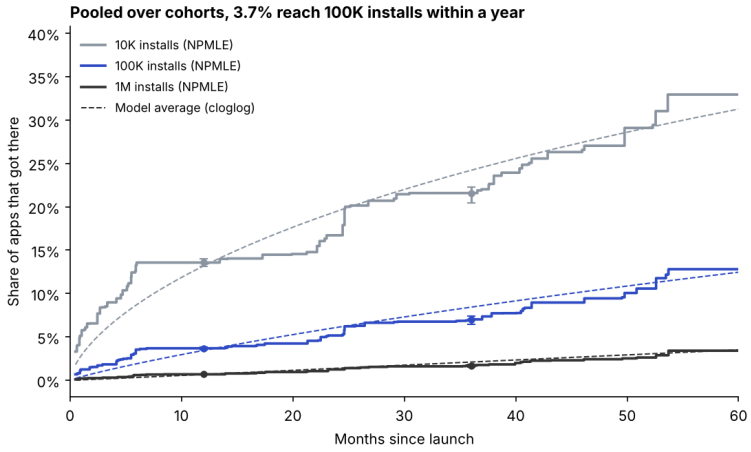


Figure 4: NPMLE of the time to 10K, 100K and 1M installs for 1,959,938 apps launched January 2016 to May 2021, with cube-root subsampling intervals at 12 and 36 months. Dashed lines are the cloglog model averaged over apps.

Figure 4 shows the pooled NPMLE curves. Within 12 months of launch 13.6% [13.1%, 14.0%] of apps reach 10K installs, 3.7% [3.5%, 3.8%] reach 100K and 0.68% [0.61%, 0.75%] reach 1M. At 36 months the numbers are 21.6%, 7.0% and 1.69%. These are pooled over launch cohorts, and they assume every cohort grows at the same pace. The curves are flat between about 6 and 12 months, which is what the estimator does when newer cohorts did better than older ones at the same age. Raw rates at a single age show the same thing. Among the 102,979 apps 11 to 13 months

old (launched mid May to mid July 2020), 13.0% had reached 10K, 3.45% had reached 100K and 0.64% had reached 1M, and the 10K share only moves between 13.0% and 14.1% across ages 5 to 16 months. The 100K value is below the pooled interval at 12 months. The subsampling intervals reflect sampling and not cohort drift. In a simulation with a known curve and 200,000 apps, intervals meant to cover 95% covered 85%.

Each model is a binary GLM fit to a random sample of 400,000 apps. Held-out log loss averaged over both checks and all milestones was 0.1972 for cloglog, 0.1993 for probit and 0.1976 for logit, so I treat the three links as a tie and use cloglog, which was lowest on the folds I scored. In the interleaved check (every fourth launch month held out) the AUC is 0.761, 0.811 and 0.867 for the three milestones and the 100K calibration slope is 0.98. In the forward check (fit on launches before June 2019, score the two years after) the AUC is 0.740, 0.797 and 0.855, and 0.754, 0.808, 0.859 within launch month. Log loss beats an age-only model in every case (forward 100K: 0.1354 vs 0.1575). The forward check under-predicts. For 100K the model averages 2.7% against a real 3.6%, and near one year of age the calibration intercepts are 0.35, 0.40 and 0.27 with slopes 1.17, 1.10 and 0.98 (Figure 5). Every app in that fit is at least 24 months old, so the check extrapolates the time curve as well as testing the no-cohort-effect assumption, and with one snapshot I cannot tell the two apart. I treat the 12-month probabilities as rough. The pooled fit is also off by age. At 5 to 7 months its 10K average is 10.4% against a raw 13.2%, and at 17 to 19 months it is 16.6% against 14.4%.

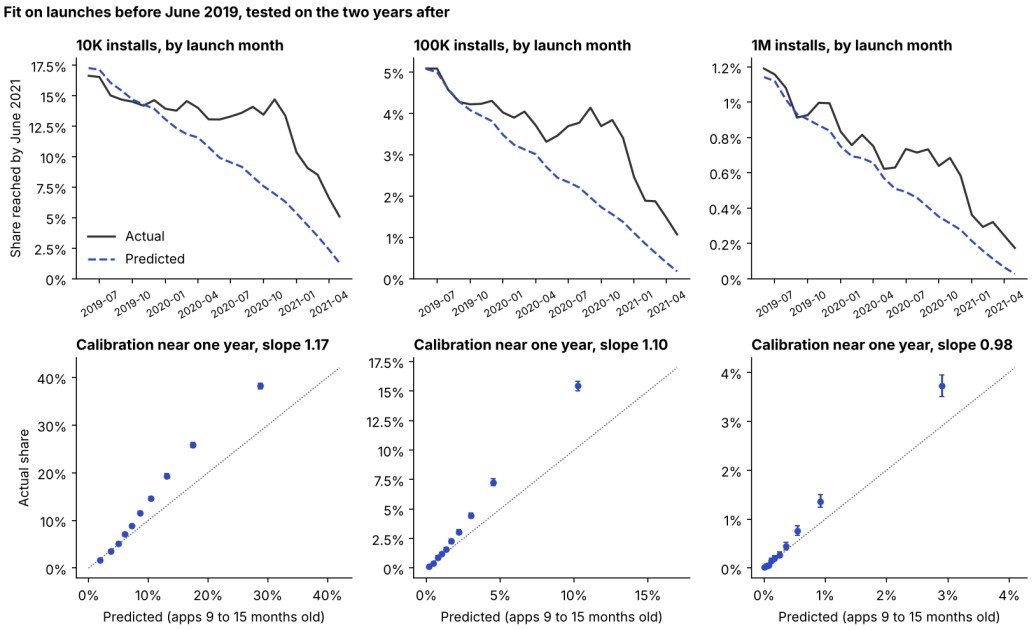


Figure 5: Forward check. Top: predicted and actual share past each milestone by launch month, for launches the model never saw. Bottom: reliability for apps 9 to 15 months old.

Intervals for one app. The scorer gives two kinds of intervals. The probability interval comes from the developer-clustered covariance and only reflects parameter uncertainty. The second is a split conformal interval for where the app will land among apps launched the same month, computed inside 20 bins of the launch score. On random held-out apps the 50%, 80% and 90% intervals covered 50.1%, 80.1% and 90.1%. Forward in time, with a score fit before June 2019, the 80% interval covered 80.7%. It covers less for games (75.2%). The intervals are wide. On the 0 to 1 scale

of launch-month percentiles the mean widths are 0.44, 0.73 and 0.84 for the 50%, 80% and 90% intervals (0.73 for 80% forward), so for one app they only give a rough neighborhood. Coverage by developer history is in Figure 6.

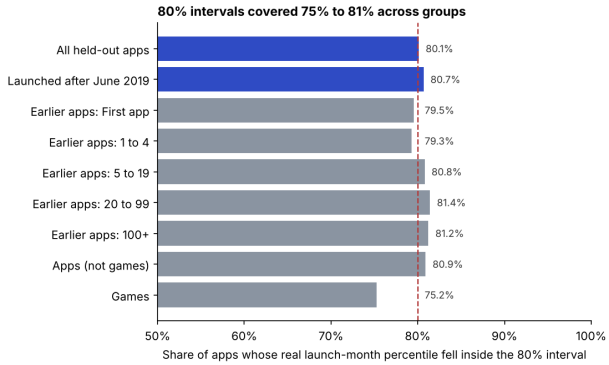


Figure 6: Empirical coverage of the 80% conformal interval.

7 Categories

Category carries most of the launch-day signal. A category-only rate scores 0.726, more than all the developer and package features together (0.711 for the strict model). Averaged over a developer's first apps of 2019 and 2020, the milestone model gives a 44.0% chance of 10K installs within a year in Simulation and 4.5% in Business. The raw rates for those developers' first apps that were 9 to 15 months old are 36.3% in Simulation (717 apps, where the model says 43.0% at those ages) and 4.5% in Business (7,628 apps, model 4.5%). Over all developers and the 48 categories, the model's average sat inside the 95% interval of the raw rate for 46% of categories at 10K and 52% at 100K, and the ratio of observed to modeled 10K ran from 0.71 to 2.09. The model is fine on average and can be off by a factor of two for one category. Across categories, the share of installs taken by the top 10 apps has a rank correlation of -0.24 with that chance. `results/study/market.csv` has demand, supply, Gini and HHI for every category.

8 Monetization: association, not effect

The old write-up found that apps with both ads and in-app purchases broke out far more often, and the simulator turned that into a lever. That reading is not supported. The flags were observed in 2021, and an app with users has every reason to add in-app purchases. Adjusting for category, launch month and developer history, apps with ads and in-app purchases in 2021 were 3.09 times as likely as ads-only apps to reach the top 5% of their launch month (95% CI 2.95 to 3.24). The E-value is 5.64 [VanderWeele and Ding, 2017], which is the risk ratio an unmeasured factor, or a reverse path from success to monetization, would need with both the flag and the outcome to explain away the whole association. I cannot rule out a reverse path that strong.

Apps never updated after release are the closest thing to a launch-day listing in this data. Among them the same ratio is 1.97 [1.55, 2.52], and among updated apps it is 2.83 [2.71, 2.96]. Hits were never updated 5% of the time against 22% for the rest, so this split is itself selected on something downstream of success. Among hits, 37% of updated apps had in-app purchases against 12% of never-updated apps, a ratio of about 3.14, which is below the 5.64 that a reverse path would need.

I read the split as a hint about timing and nothing stronger. None of these numbers is the effect of choosing a business model, and the next version of the simulator should not present them as levers.

9 Limitations

- Survivorship. Only apps listed in June 2021 are observed. Older cohorts have lost more failures, which biases the long-horizon parts of the milestone curves upward and makes developer history run low.
- One snapshot. Training labels for the 2018 cohort were also measured in 2021, so a true backtest is not possible and the temporal split is the closest I can get. The milestone model's no-cohort-effect assumption can only be checked out of sample, and the forward check under-predicts. The pooled curves and their intervals ignore cohort drift.
- Category and title are 2021 values. I treat them as launch information because developers pick them before launch, but either can be edited. The strict model without them scores 0.711, and dropping the title features alone lowers the ablation AUC from 0.830 to 0.783.
- The probability intervals are parameter intervals. They don't cover model error. The conformal intervals do have checked coverage, but only for the launch-month percentile, and they cover less for games.
- The data covers Google Play only, and nothing after June 2021.

10 Reproducibility

Everything runs from `make all` after `bash get_data.sh`. Every number in this paper is written to `numbers.tex` by `paper/make_numbers.py` from the saved results. Tests (`make test`) run the full pipeline on a small synthetic dataset, check the metric code against scikit-learn, check the survival estimators on data with a known answer, and check that the exported model reproduces the fitted model to 10^{-9} in Python and JavaScript.

References

P. Groeneboom and J. A. Wellner. *Information Bounds and Nonparametric Maximum Likelihood Estimation*. Birkhäuser, 1992.

D. N. Politis and J. P. Romano. Large sample confidence regions based on subsamples under minimal assumptions. *Annals of Statistics*, 22(4), pages 2031 to 2050, 1994.

G. Prakash. Google Play Store dataset (2.3M+ apps). GitHub, 2021.

B. Sen, M. Banerjee and M. Woodroofe. Inconsistency of bootstrap: the Grenander estimator. *Annals of Statistics*, 38(4), pages 1953 to 1977, 2010.

T. J. VanderWeele and P. Ding. Sensitivity analysis in observational research: introducing the E-value. *Annals of Internal Medicine*, 167(4), pages 268 to 274, 2017.